FRANKFURT UNIVERSITY OF APPLIED SCIENCES

Faculty 2: Computer Science and Engineering

Computer Science Department

Project Report

BOOKSTORE ADMINISTRATION

Module: Programming Exercises

1. Tuan Viet Nguyen - 1617863
2. Tieu Cuong Luong - 1616329
3. Nguyen Thien Nguyen - 1615629
4. Hoang Yen Ngoc Nguyen - 1614693

Instructor: Lecturer Ralf-Oliver Mevius

Frankfurt am Main, SS2025

Evaluation Form

Tuan Viet Nguyen - 1617863

Tieu Cuong Luong - 1616329

Nguyen Thien Nguyen - 1615629

Hoang Yen Ngoc Nguyen - 1614693

Evaluation

Note

Frankfurt am Main, 27/06/2025

Signature/lecturer(s)

Tables of Contents

List of Abbreviations

ACID	Atomicity, Consistency, Isolation, Durability *(acronym)*
AJAX	Asynchronous JavaScript & XML
API	Application Programming Interface
CI/CD	Continuous Integration/Continuous Deployment
CRUD	Create, Read, Update, Delete *(acronym)*
CSS	Cascading Style Sheets
e.g.	For example
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ID	Identity Document
IDE	Integrated Development Environment
JDBC	Java Database Connectivity
JDK	Java Development Kit
JPA	Jakarta Persistence API (formerly Java Persistence API)
JSON	JavaScript Object Notation
MVC	Model-View-Controller
ORM	Object Relational Mapping
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer
SEO	Search Engine Optimization
SLF4J	Simple Logging Facade for Java
SQL	Structured Query Language
UI/UX	User Interface/User Experience
URL	Uniform Resource Locator
WAR	Web Application Resource
WCAG	Web Content Accessibility Guidelines

List of Figures

List of Tables

List of Attachments

Contents

1. INTRODUCTION & PROJECT OVERVIEW

In the modern era of digital commerce, online bookstores have become an essential platform for readers to access a wide range of books conveniently from any location. Traditional brick-and-mortar bookstores face limitations in terms of physical space, inventory management, and operational costs. An efficient online bookstore administration system addresses these challenges by offering an organized, scalable, and user-friendly digital solution. This project, the Bookstore Administration, aims to simulate such an environment, providing both customers and administrators with a comprehensive set of features to manage book sales, inventory, and customer orders efficiently. The system demonstrates how technology can streamline the book purchasing experience, reduce manual workload for store operators, and open opportunities for business scalability.

1.1. Scope of the system

The Bookstore Administration is a web-based application designed to manage the core operations of an online bookstore. The system provides features for both customers and administrators. Customers can browse, search, and purchase books, while administrators can manage the inventory and monitor order status. The system supports user registration, authentication, shopping cart management, order processing, and order history tracking. It includes a backend implemented in Java using the Spring Boot framework, a relational database in MySQL for persistent storage, and a frontend with web templates and static resources.

The system exposes RESTful APIs that allow interaction between the frontend and backend, enabling potential future integration with other platforms such as mobile applications or third-party services.

1.2. Business Process Overview

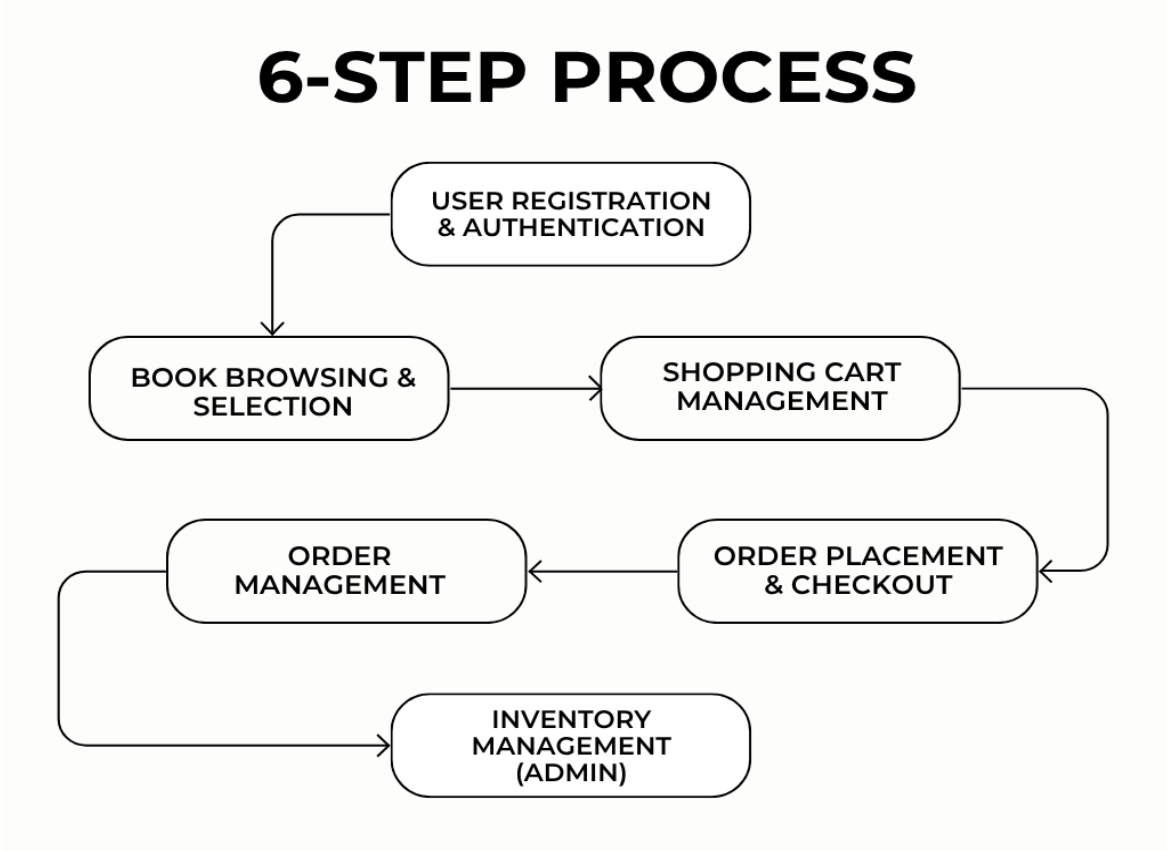

Image 1. Business Process Overview

The bookstore business process includes the following steps:

1. User Registration and Authentication:
    - New users register accounts.
    - Registered users log in to access personalized features.
2. Book Browsing and Selection:
    - Users browse available books.
    - Book listings show title, author, price, and availability.
3. Shopping Cart Management:

- Users add books to their shopping cart.
- The cart maintains items until checkout.

4. Order Placement and Checkout:
   - Users proceed to checkout and place orders.
   - The system calculates the total and processes the order.
5. Order Management:

- Users view their order history and details.
- Administrators manage all orders.

6. Inventory Management (Admin):

- Administrators add, update, or remove books.
- The system updates stock levels accordingly.

1.3. Project Objectives

1.3.1. Functional Requirements

Users' Functions:

- Register and create an account.
- Log in and authenticate.
- Browse, search, and view detailed information about books.
- Add, update, and remove books from the shopping cart.
- Complete the checkout process to place orders.
- View personal order history and order details.

Administrator's Functions:

- Login with administrator credentials.
- Add new books to the inventory.
- Update existing book information.
- Remove books from the inventory.
- View and manage all customer orders.
- Monitor inventory stock levels.

1.3.2. Non-Functional Requirements

- Usability:

To optimize user experience, the system must feature an intuitive interface and clear instructions, ensuring users can learn and operate it easily. Navigation menus have to be clearly structured, allowing quick access to features like browsing books, managing the cart, and accessing user/admin functionalities. The system is also expected to provide real-time feedback for all actions, including confirmation of items added to the cart, successful order placement, and error messages, ensuring users have transparent insight into operational status.

- Security:

To prevent unauthorized access, the system requires secure authentication. This protects sensitive data, including user credentials and order information. Additionally, session checks are mandatory before processing any orders or cart actions, ensuring the legitimacy of each interaction.

- Reliability:

The system is designed for error-free operation during normal usage, ensuring core functions perform accurately over extended periods. Database constraints are implemented to prevent orphaned records, maintaining data integrity. In the event of an issue, error messages are compulsory to guide users toward resolution.

- Performance:

The system should deliver fast response times for all key user interactions. This ensures a smooth and responsive user experience, minimizing wait times and keeping users engaged throughout their shopping journey.

- Availability:

The system must prioritize operational continuity and accessibility, aiming for minimal downtime. Core services should be available during business hours or as required, with 24/7 user access assuming server uptime. During normal usage, session and cart data are preserved. Administrators can effectively monitor and manage the system through the dedicated admin dashboard.

- Scalability:

The system aims to support growth in users, books, and transactions without performance degradation. Its modular architecture facilitates the easy addition of new features and integration with external services. The backend and database are facilitated for scalability, accommodating an increasing user base. Furthermore, the codebase should be ready for deployment in distributed/cloud environments.

2. SYSTEM ANALYSIS & DESIGN

2.1. System Analysis

2.1.1. Stakeholders Analysis

- Customers:

Role: The ultimate end-users and primary beneficiaries of the system.

Responsibility: Directly interact with the system to register and manage their personal accounts, as well as browse, search, and select books. They are responsible for managing their shopping carts and completing the order placement process, while also being able to track and review their order history. Furthermore, customers provide crucial feedback and report any issues, which helps improve service quality.

- Administrators:

Role: Operate, monitor, and maintain the stability of the system from the provider's side.

Responsibility: Ensure the smooth operation of the platform by comprehensively managing book inventory (adding, updating, and deleting book information). They directly oversee and process customer orders and are responsible for maintaining the accuracy of system data. Additionally, administrators play a vital role in providing customer support and resolving operational issues as they arise.

- System Developers:

Role: Design, build, and evolve the technical components of the system.

Responsibility: Handle the design, implementation, and maintenance of system functionalities, including the development of both frontend user interface modules and backend server-side logic. They ensure the code quality, scalability, and maintainability of the entire system, while also performing thorough system testing and debugging to ensure stable and efficient operation.

- Database Administrators:

Role: Manage and optimize the project's data infrastructure.

Responsibility: Ensure the efficient operation of the database by designing and managing its schema. They are primarily responsible for ensuring data integrity, performing backups, and executing data recovery when necessary. Concurrently, database administrators focus on optimizing database performance to guarantee rapid data access and processing.

- Project Manager:

Role: Coordinate, supervise, and hold overall accountability for the project's success.

Responsibility: Serve as the central figure in detailed planning and overseeing the entire project execution process. They are responsible for effectively coordinating among the various stakeholder groups, from development to operations. The Project Manager's primary objective is to ensure that project objectives, scope, and deadlines are met rigorously and successfully.

Building upon the identification of key stakeholders, we now transition to a more detailed representation of system functionality. This section presents the **Use Case Diagram**, specifically focusing on the interactions between the Customers and Administrators. This diagram visually outlines the distinct actions and system capabilities accessible to each of these primary actors.

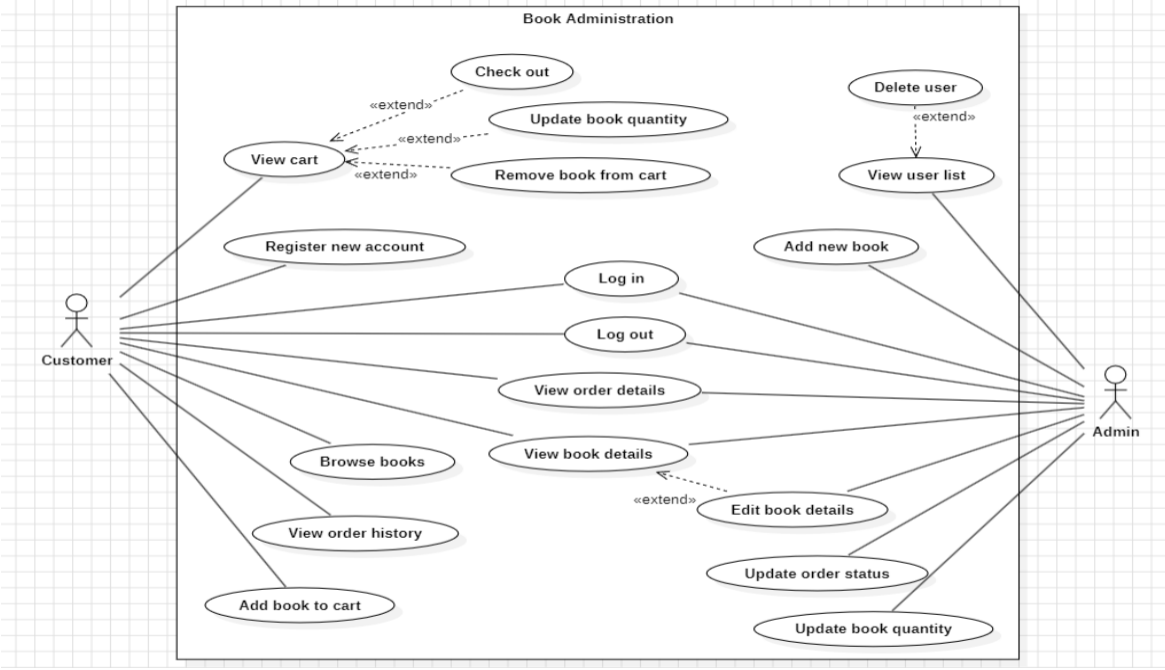

Figure 1. Primary Actors Use Case Diagram

Following the Use Case Diagram, the subsequent Use Case Actions will detail the precise steps and system responses for each identified use case, elaborating on the specific interactions for these key actors.

Use case/ Actors	Customer	Admin
Log in	Authenticate to access personalized features	Authenticate with admin credentials
Log out	End their session	End their session
View book details	Observe detailed information about a specific book	Observe detailed information for inventory management
View order details	Observe the specific books and quantities for a past order	View details of all orders
Browse books	View available books, filtered by	

	type, search by name or publisher	
Register new account	Create a new user account	
View cart	View all items currently in their cart	
Add book to cart	Add a selected quantity of a book in the shopping cart	
Update book quantity	Change the quantity of a book in the shopping cart	
Remove book from cart	Delete a book from the shopping cart	
Check out	Place an order for items in the shopping cart	
View order history	Observe a list of their past orders	
Add new book		Add a specific books into inventory
Edit book details		Update information such as title, author, price
Update book quantity		Adjust stock levels for each book
View user list		View all users existing in the system
Update order status		Change the status of an order (e.g., "Pending" to "Shipped")
Delete user		Delete user accounts

Table 1. Primary Actors Use Case Actions

2.1.2. Business Process Modeling

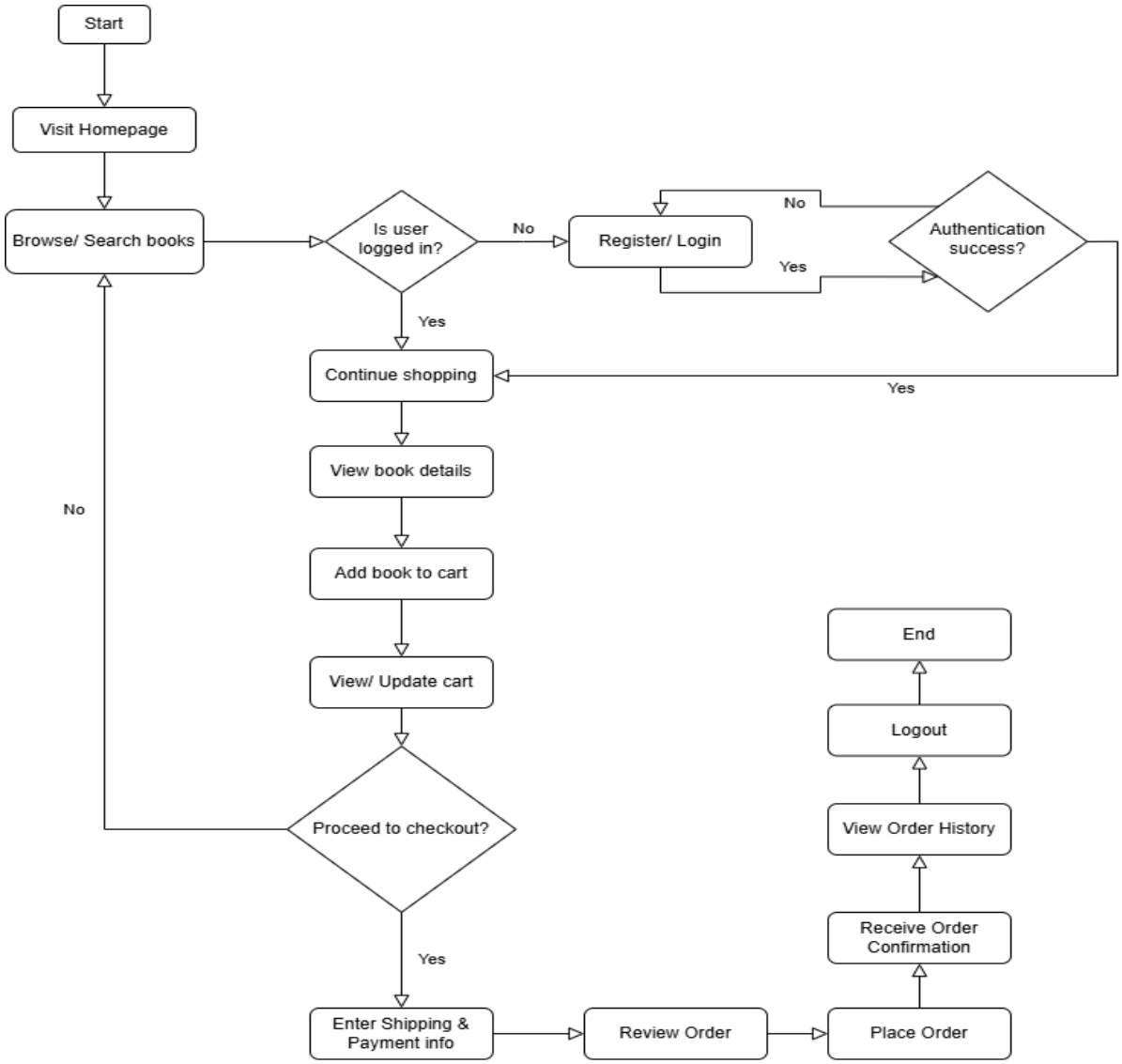

Figure 2. User Shopping Journey Activity Diagram

- User Shopping Journey:

A user's shopping journey on the bookstore platform begins on the homepage, allowing them to browse or search for books without logging in. Detailed information, including title, author, price, and availability, is accessible for each book. To add items to their shopping cart or make a purchase, users are prompted to log in or register. Upon successful authentication, they gain access to personalized features and can resume their shopping experience.

Users can update their shopping cart at any time by modifying quantities or removing items. When ready to purchase, they proceed to checkout, where they provide shipping and payment information. After reviewing their order for accuracy, the user confirms it, and the system processes the order, generating an order confirmation.

Following a successful purchase, users can access their order history, view detailed information about past orders. The shopping session concludes when the user logs out of the system, ensuring a seamless and secure experience from browse to order completion.

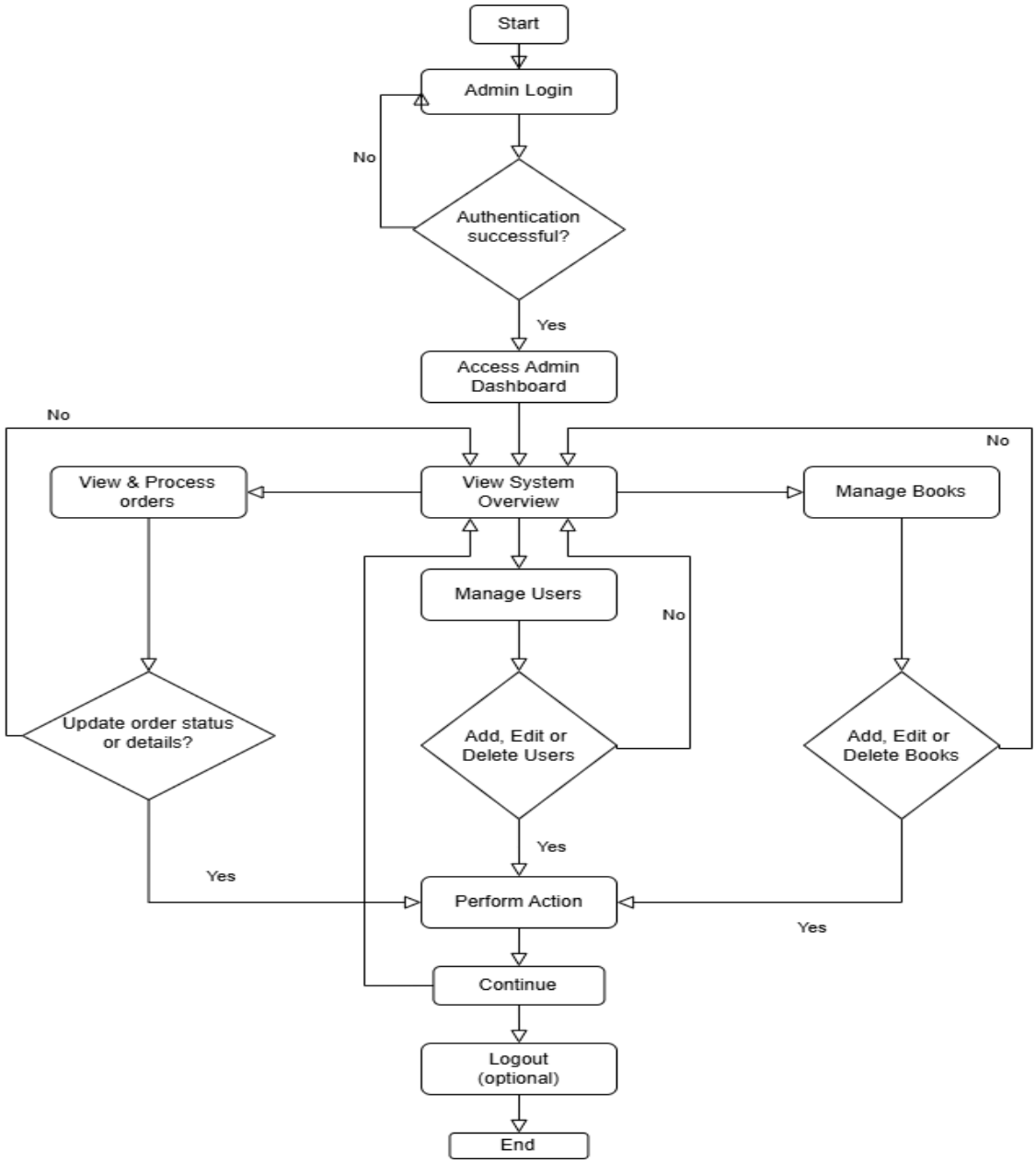

Figure 3. Admin System Management Activity Diagram

- Admin System Management:

The activity flow for an admin in the bookstore administration system begins with the login process. The admin initiates the session by providing login credentials, which are then authenticated by the system. If authentication fails, the admin is prompted to retry; upon

successful authentication, the admin gains access to the Admin Dashboard, which serves as the central point for all subsequent management activities.

Once inside the Admin Dashboard, the admin can view the System Overview, which offers options to manage users, manage books, or process orders. If the admin chooses to manage users, they are directed to a function that allows them to add, edit, or delete user accounts. Similarly, selecting the manage books option enables the admin to add, edit, or delete book records. In both scenarios, if any modifications are made, the admin performs the action and is then returned to the main workflow to continue operations.

In the order processing section, the admin can view and manage orders. If an update to the order status or details is necessary, the admin executes the appropriate changes before continuing. After completing any management or processing tasks, the admin may choose to log out, concluding the session. This structured workflow ensures a systematic and controlled approach to administering the bookstore's operations, safeguarding both data integrity and efficient task management.

2.2. System Design

2.2.1. System Architecture Overview

2.2.1.1. Client-Server Architecture

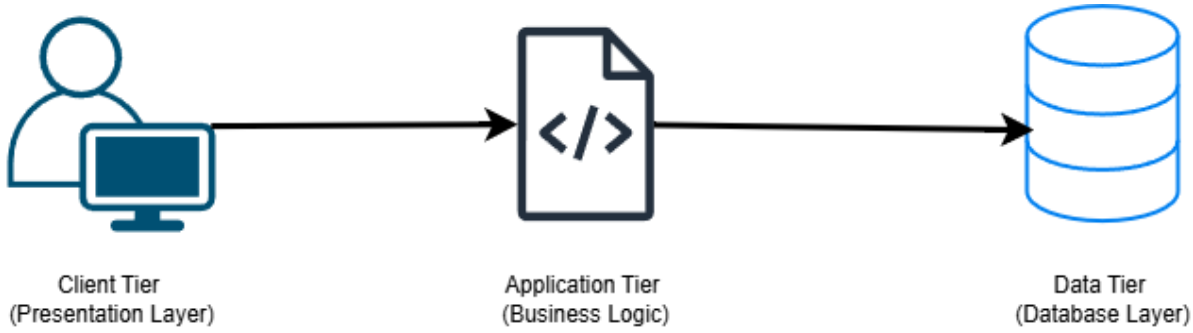

Image 2. 3-tier Client-Server Architecture

The Bookstore Administration System employs a three-tier client-server architecture that strategically separates concerns across distinct layers, promoting modularity, scalability, and maintainability through clear component boundaries and responsibilities.

The Client Tier operates within web browsers and handles all user interface rendering through server-generated dynamic HTML content using Thymeleaf templating. Client-side JavaScript manages interactive behaviors and AJAX communications, while CSS provides visual styling and responsive design elements.

The Application Tier serves as the processing engine, implemented as a Spring Boot application that orchestrates all business operations. This tier encompasses request routing through controllers, business logic execution, and data access coordination through specialized DAO components. It maintains application state, enforces business rules, and manages the complete request-response lifecycle between clients and data storage.

The Data Tier provides robust persistent storage through a MySQL relational database system that securely maintains all application entities including user accounts, book catalog, shopping carts, and order histories. This layer ensures data integrity, supports concurrent access patterns, and executes complex queries required by the business logic tier.

This approach promotes independent scaling of different system components based on performance requirements and usage patterns. The clear separation allows development teams to work on different layers simultaneously while maintaining consistent interfaces. Additionally, the modular design supports future technology migrations, such as database

platform changes or presentation layer enhancements, without requiring comprehensive system redesign.

2.2.1.2. Component Integration & Technologies

The architectural implementation leverages complementary technologies that integrate seamlessly across tiers:

- Presentation Layer: utilizes Spring MVC framework with Thymeleaf templating engine to generate server-side rendered pages that provide immediate content delivery and SEO-friendly markup.

- Business Logic Layer: employs Java with Spring Framework annotations for dependency injection, transaction management, and aspect-oriented programming capabilities that simplify complex enterprise patterns.

- Data access operations are abstracted through Spring Data JPA with Hibernate implementation, providing object-relational mapping capabilities that translate business objects into database operations while maintaining database vendor independence.

- MySQL Database Layer: supports ACID transactions, referential integrity constraints, and optimized query execution that ensures reliable data management under concurrent user loads.

2.2.1.3. API Design

The Bookstore Administration System employs a RESTful API architecture that serves as the communication layer between the frontend user interface and the backend business logic. This design provides a standardized, scalable approach to handling all system operations through well-defined service interfaces.

- API Architecture Principles

The system's API design is built upon three fundamental principles that ensure consistency, scalability, and maintainability.

The first principle is resource-based design, where APIs are organized around core business `@Entities` including `Book`, `User`, `Order`, and `ShoppingCart`. Each resource follows a clear, hierarchical URL structure that adheres to REST conventions, creating predictable endpoint patterns that facilitate both development and integration processes.

The second principle involves proper HTTP Mapping, where each operation type is associated with its corresponding HTTP verb. `GET` requests are utilized for data retrieval operations such as viewing books, accessing cart contents, and fetching order information. `POST` requests handle the creation of new resources including adding books to the catalog, placing orders, and user registration processes. `PUT` requests manage updates to existing resources such as modifying book details and updating order statuses, while `DELETE` requests handle resource removal operations such as deleting books from the catalog and removing items from shopping carts.

The third principle emphasizes stateless communication, ensuring that each API request contains all necessary information for processing without relying on server-side state. Session management is handled through secure tokens and cookies, eliminating dependencies between consecutive requests and enabling horizontal scaling capabilities.

- Core API Categories

The API structure is organized into four primary service categories that collectively support all business operations within the bookstore system.

Authentication and Authorization Services form the foundation of the system's security model, providing comprehensive user registration and login endpoints alongside robust session management and validation mechanisms. These services implement role-based access control to distinguish between customer and administrative privileges while ensuring secure logout and session termination processes.

Catalog Management Services encompass all book-related operations, offering sophisticated book retrieval capabilities with advanced filtering and search functionalities. These services

support category-based browsing to enhance user experience and provide complete administrative book management through CRUD operations. Additionally, they maintain real-time inventory tracking and stock management to ensure accurate availability information across the system.

Shopping Cart Services address the dual nature of the system's user base by supporting both guest and authenticated user interactions. For guest users, the services provide session-based cart management, while authenticated users benefit from database-backed cart persistence. A critical feature of these services is the seamless cart synchronization mechanism that transfers guest cart contents to user accounts upon authentication, ensuring no loss of shopping progress during the transition from anonymous to authenticated states.

Order Processing Services complete the e-commerce workflow by facilitating order creation from cart contents and maintaining comprehensive order history retrieval capabilities. These services provide real-time order status tracking and updates for customers while offering administrative order management tools for system operators. The integration between cart and order services ensures a smooth transition from shopping to purchasing, maintaining data integrity throughout the entire transaction process.

2.2.1.4. Data Security & Privacy

The Bookstore Administration System implements several basic security measures to protect user data and system integrity:

Session-Based Authentication: The system uses HTTP sessions to manage user authentication state. User login status is tracked through session attributes including username and role, which are validated before accessing protected resources. A `Login` controller and `Register` controller handle user authentication by verifying credentials against the database and establishing session state upon successful login.

Role-Based Access Control: The system implements basic role-based access control where users are assigned either "customer" or "admin" roles. A proposing `Admin` controller is

implemented to check for admin role in the session before allowing access to sensitive operations such as adding books, updating inventory, or deleting books. This prevents regular customers from accessing administrative features.

Input Validation: Basic input validation is implemented across the controllers. The registration process validates that required fields are not empty and enforces minimum password length requirements. The admin book management functions validate that required book information is provided before processing requests. API endpoints include null checks and basic data type validation before processing user inputs.

Cross-Origin Resource Sharing (CORS) Configuration: A web configuration class that includes CORS configuration needs to be implemented to allow all origins, methods, and headers. While this provides broad access for development purposes, it represents an area where access controls are configured at the application level.

Error Handling Without Information Disclosure: The system should implement try-catch blocks throughout the codebase that log detailed error information for debugging while returning generic error messages to users. This prevents the exposure of internal system details, database schema information, or other sensitive technical details to end users.

Data Access Layer Separation: This pattern is responsible for implementation and provides separation between business logic and database operations. While the current implementation doesn't use parameterized queries explicitly shown in the visible code, the DAO layer abstracts database access and centralizes data operations, which supports secure database interaction patterns.

Session State Validation: Controllers consistently check for valid user sessions before processing requests that require authentication. Unauthenticated users are redirected to login pages, and operations requiring specific roles validate the user's role from the session before proceeding.

Guest Cart to User Cart Security: The system includes functionality to securely transfer guest shopping cart data to authenticated user accounts upon login, ensuring that cart data is properly associated with the correct user and preventing cart data leakage between users.

2.2.1.5. Module Design

The Bookstore Administration follows a modular architecture pattern that promotes separation of concerns, maintainability, and scalability. The system is structured into distinct modules, each responsible for specific business functions and technical concerns, ensuring clear boundaries and well-defined interfaces between components.

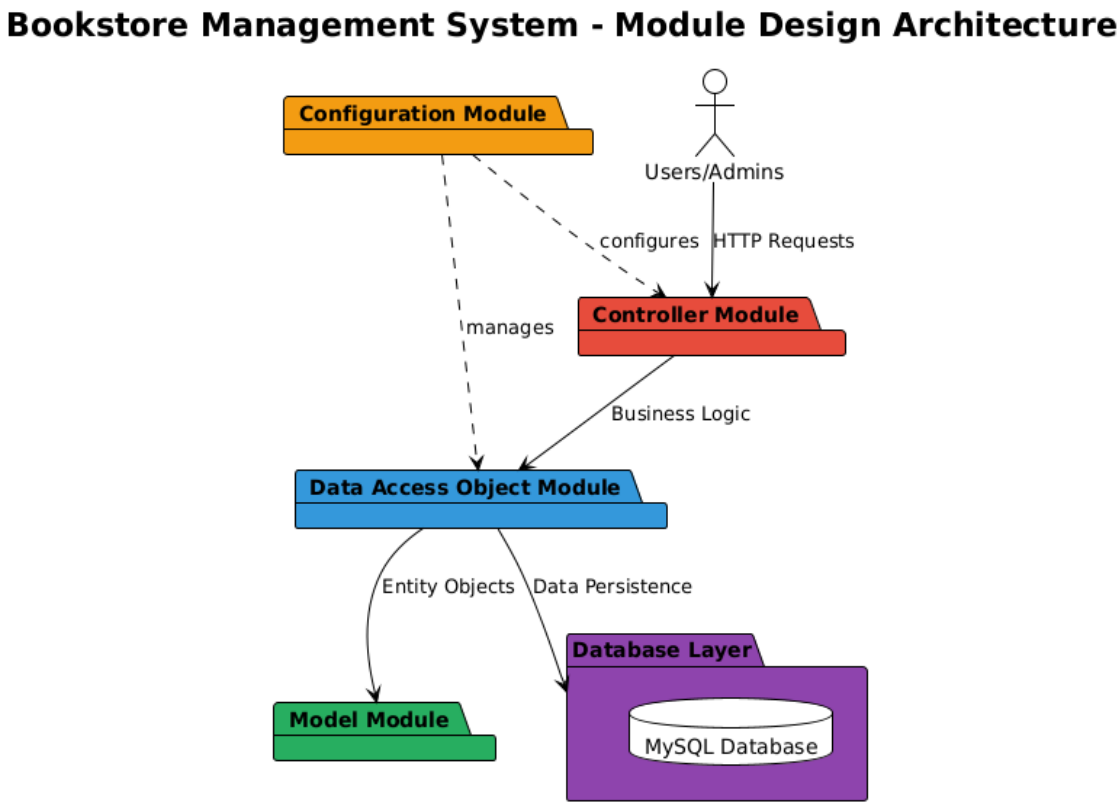

Image 3. Module Design Architecture

MODEL Module

The Model module serves as the foundation of the system's data representation layer, containing entity classes that correspond directly to the database schema. This module encapsulates the core business objects including `User`, `Book`, `Order`, `OrderDetails`, and `ShoppingCart` entities. Each model class defines the structure and attributes of business data, implementing proper encapsulation through private fields and public accessor methods. The model classes also incorporate validation logic and business rules that govern data integrity, ensuring that objects maintain consistent states throughout their lifecycle. This module operates independently of other system components, promoting reusability and enabling easy modification of data structures without affecting business logic or presentation layers.

Data Access Object Module

The DAO module implements the data persistence layer, providing a clean abstraction between the business logic and the underlying database operations. This module contains specialized classes such as `UserDAO`, `BookDAO`, `OrderDAO`, `OrderDetailDAO`, and `ShoppingCartDAO`; each responsible for managing database interactions for their respective entities. The DAO pattern encapsulates all SQL operations, connection management, and result set processing, shielding the business logic from database-specific implementation details. Each DAO class provides standardized CRUD operations while also offering specialized query methods tailored to specific business requirements. This design facilitates database technology changes and enables comprehensive testing through mock implementations, while maintaining transaction integrity and optimal performance through connection pooling and prepared statements.

CONTROLLER Module

The Controller module implements the presentation and API layer, serving as the primary interface between external clients and the system's business logic. This module contains multiple controller classes including `BookApiController`, `AdminController`, `LoginController`, `RegisterController`, `CartController`, `OrderController`,

and `CheckoutController`; each handling specific functional areas of the application. The controllers are responsible for request routing, input validation, session management, and response formatting, ensuring proper separation between user interface concerns and business processing. They implement RESTful API endpoints that support both traditional web interactions and modern AJAX-based single-page applications. The controller design incorporates comprehensive error handling, logging, and security measures including role-based access control and input sanitization.

CONFIGURATION Module

The Configuration module manages system-wide settings and framework integration, defines application behavior and external service integration. This module handles cross-cutting concerns including CORS configuration for API access, static resource management for web assets, and security policy enforcement. The configuration classes utilize Spring Boot's annotation-driven approach to minimize boilerplate code while providing flexible customization options for different deployment environments. This centralized configuration approach ensures consistent behavior across all system components and simplifies maintenance and deployment processes.

2.2.2. Data Model Description

The system utilizes several data models (Java classes) to represent the core entities and their relationships. These models are designed to map to database tables using JPA annotations.

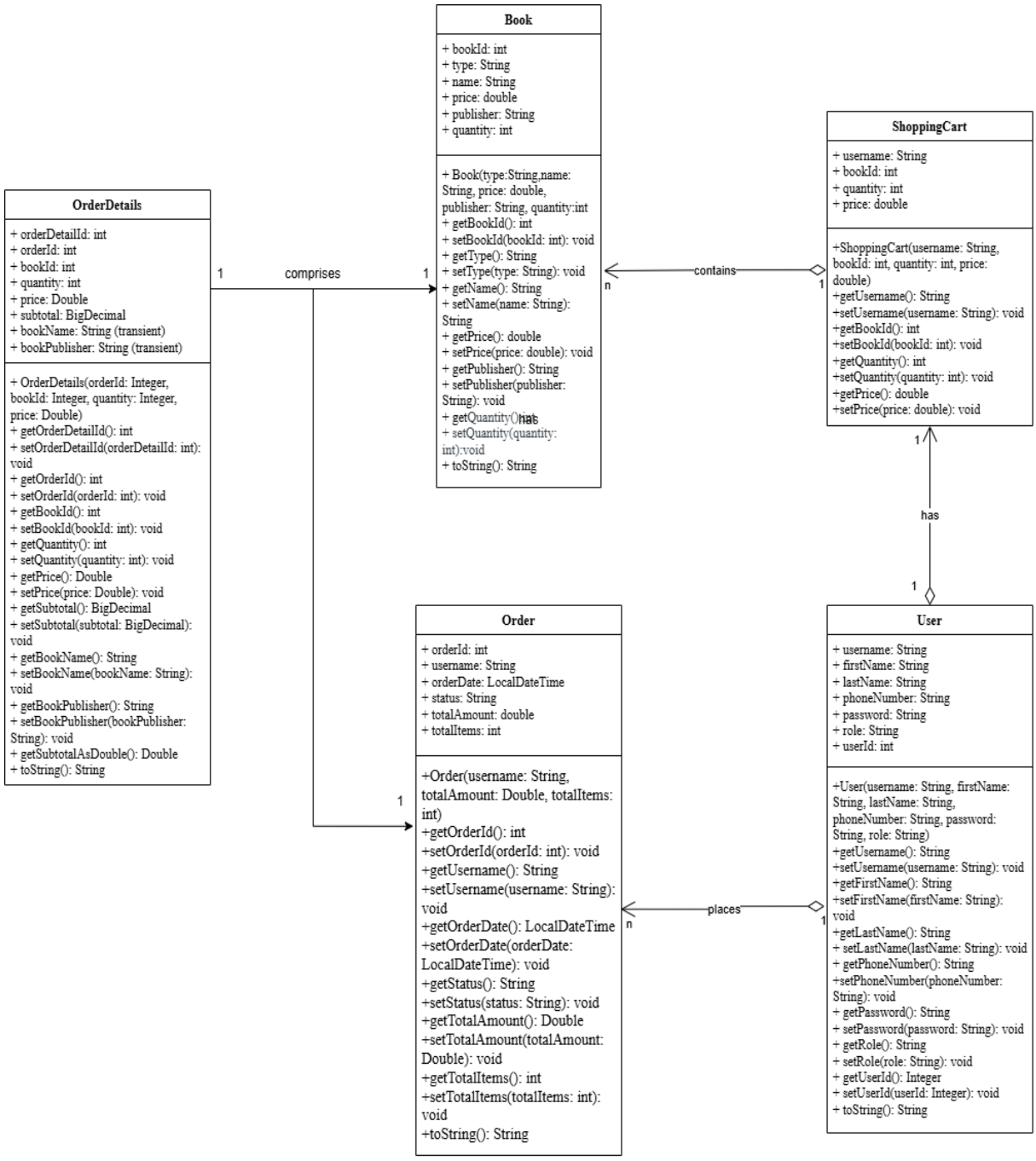

Figure 4. Data Model Class Diagram

Based on the data model class diagram shown in Figure 4, the table below provides a detailed description of the corresponding classes and their attributes.

Class Name	**Attributes (Type)**	**Key Methods**	**Description**
Book	bookId (Integer), type (String), name (String), price (Double), publisher (String), quantity (Integer)	Getters & Setters for all attributes, toString()	Represents a single book in the system. It holds details such as its unique ID, genre, title, price, publisher, and current stock quantity. This class maps directly to the books table in the database.
User	username (String), firstName (String), lastName (String), phoneNumber (String), password (String), role (String), userId (Integer)	Getters & Setters for all attributes, toString()	Represents a user of the system, who can be either a customer or an admin. It stores user credentials (username, password), personal information, contact details, and their role, which dictates system permissions. This class maps to the users table.

Order	orderId (Integer), username (String), orderDate (LocalDateTime), status (String), totalAmount (Double), totalItems (Integer)	Getters & Setters for all attributes, toString()	Represents a customer's placed order. It contains information about the unique order ID, the username of the placing user, the date and time of the order, its current processing status, the totalAmount paid, and the totalItems (distinct books) in the order. This maps to the orders table.

OrderDetails	orderDetailId (Integer), orderId (Integer), bookId (Integer), quantity (Integer), price (Double), subtotal (BigDecimal), bookName (String - Transient), bookPublisher (String - Transient)	Getters & Setters for all attributes, getSubtotalAsDouble() , toString()	Represents a specific line item within an Order. Each entry links an Order to a Book, specifying the quantity of that book purchased and its price at the time of purchase. It also calculates a subtotal for the line. bookName and bookPublisher are transient fields for display convenience, not stored in the database. This maps to the orderdetails table.

ShoppingCart	username (String), bookId (Integer), quantity (Integer), price (Double)	Getters & Setters for all attributes	Represents an item in a user's shopping cart before checkout. It uses a composite primary key formed by username and bookId to uniquely identify each item in a user's cart. It stores the quantity of the book and its price when added to the cart. This maps to the shoppingcart table.

Table 2. Data Model Classes Description

2.2.3. Database Design

The database design reflects the data models, with tables created for each persistent entity. Relationships are enforced using primary and foreign keys.

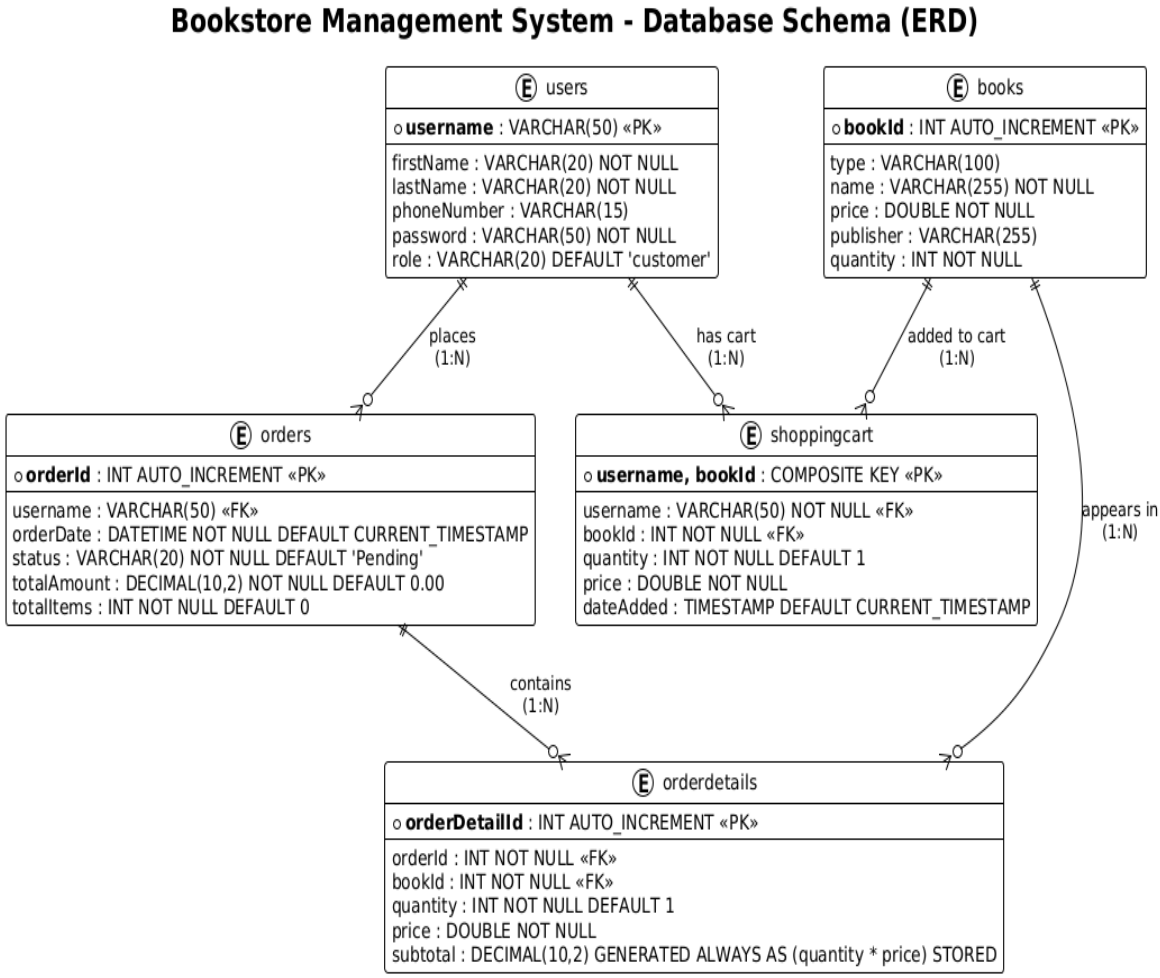

Figure 5. Database ER Diagram

The following code reflects the database structure outlined in the ER diagram, with explanations for each table and relationship.

```
-- DROP DATABASE and RECREATE (Clean Slate Approach)

DROP DATABASE IF EXISTS bookstore;

CREATE DATABASE bookstore;

USE bookstore;



--
```

```
-- Table structure for table `books`
--


CREATE TABLE `books` (
  `bookId` int NOT NULL AUTO_INCREMENT,
  `type` varchar(100) DEFAULT NULL,
  `name` varchar(255) NOT NULL,
  `price` double NOT NULL,
  `publisher` varchar(255) DEFAULT NULL,
  `quantity` int NOT NULL,
  PRIMARY KEY (`bookId`)
) ENGINE=InnoDB AUTO_INCREMENT=100038 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci;


--
-- Dumping data for table `books`
--


INSERT INTO `books` VALUES
(100000,'Fiction','To Kill a Mockingbird',18.99,'J.B. Lippincott & Co.',8),
(100001,'Fiction','1984',14.99,'Secker & Warburg',0),
(100002,'Fiction','Pride and Prejudice',12.99,'T. Egerton',50),
(100003,'Fiction','The Great Gatsby',10.99,'Charles Scribner\'s
Sons',197),
(100004,'Fiction','Moby-Dick',16.5,'Harper & Brothers',11),
(100005,'Science-Fiction','Dune',15.5,'Chilton Books',72),
(100006,'Science-Fiction','Neuromancer',13.99,'Ace',10),
(100007,'Science-Fiction','Fahrenheit 451',22,'ddfasd',12),
```

```
(100008,'Science-Fiction','The Left Hand of Darkness',12.5,'Ace',5),

(100009,'Science-Fiction','Ender\'s Game',14.99,'Tor Books',9),

(100010,'Fantasy','Harry Potter and the Sorcerer\'s Stone',20,'Bloomsbury',15),

(100011,'Fantasy','The Hobbit',14.99,'George Allen & Unwin',20),

(100012,'Fantasy','The Name of the Wind',17.99,'DAW Books',12),

(100013,'Fantasy','American Gods',16.5,'William Morrow',9),

(100014,'Fantasy','Mistborn: The Final Empire',15.5,'Tor Books',7),

(100015,'Non-Fiction','Sapiens: A Brief History of Humankind',22.99,'Harvill Secker',12),

(100016,'Non-Fiction','Educated',19.99,'Random House',10),

(100017,'Non-Fiction','Becoming',24.99,'Crown',14),

(100018,'Non-Fiction','The Immortal Life of Henrietta Lacks',16.99,'Crown',100),

(100019,'Fiction','Thinking, Fast and Slow',18.5,'Farrar, Straus and Giroux',111),

(100020,'Biography','The Diary of a Young Girl',10.99,'Contact Publishing',5),

(100021,'Biography','Steve Jobs',25,'Simon & Schuster',10),

(100022,'Biography','Long Walk to Freedom',18.99,'Little, Brown and Company',6),

(100023,'Biography','Becoming',24.99,'Crown',8),

(100024,'Biography','The Wright Brothers',14.99,'Simon & Schuster',4),

(100026,'Manga','One Piece',10.99,'Shueisha',1000),

(100027,'Manga','Attack on Titan',11.5,'Kodansha',18),

(100028,'Manga','My Hero Academia',8.99,'Shueisha',222);


--

-- Table structure for table `users`
```

```
--


CREATE TABLE `users` (
  `username` varchar(50) NOT NULL,
  `firstName` varchar(20) NOT NULL,
  `lastName` varchar(20) NOT NULL,
  `phoneNumber` varchar(15) DEFAULT NULL,
  `password` varchar(50) NOT NULL,
  `role` varchar(20) DEFAULT 'customer',
  PRIMARY KEY (`username`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;


--
-- Dumping data for table `users`
--


INSERT INTO `users` VALUES
('admin','Harry','Vu','1234567899','adminpass','admin'),
('alice','Alice','Smith','1234567890','password123','customer'),
('bob','Bob','Johnson','2345678901','bobsecure','customer'),
('nghia123','Harry','Vu','01234224782','123456789','customer'),
('khoa','Khoa Le','Tran','44115487211','00000000','customer'),
('khoa123','Khoa','Nguyen','087159158','0123456789','customer'),
('phamthithi','Thi','Thi','0123456789','0123456789','customer');


--
-- Table structure for table `orders` (WITH ALL REQUIRED COLUMNS)
```

```
--


CREATE TABLE `orders` (
  `orderId` int NOT NULL AUTO_INCREMENT,
  `username` varchar(50) DEFAULT NULL,
  `orderDate` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `status` VARCHAR(20) NOT NULL DEFAULT 'Pending',
  `totalAmount` DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  `totalItems` INT NOT NULL DEFAULT 0,
  PRIMARY KEY (`orderId`),
  KEY `idx_orders_username` (`username`),
  KEY `idx_orders_date` (`orderDate`),
  CONSTRAINT `fk_orders_username` FOREIGN KEY (`username`) REFERENCES
`users` (`username`) ON DELETE SET NULL
)    ENGINE=InnoDB    AUTO_INCREMENT=200018    DEFAULT    CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci;


--
-- Table structure for table `orderdetails` (WITH ALL REQUIRED COLUMNS)
--


CREATE TABLE `orderdetails` (
  `orderDetailId` int NOT NULL AUTO_INCREMENT,
  `orderId` int NOT NULL,
  `bookId` int NOT NULL,
  `quantity` int NOT NULL DEFAULT 1,
  `price` double NOT NULL,
  `subtotal` DECIMAL(10,2) GENERATED ALWAYS AS (quantity * price) STORED,
```

```
  PRIMARY KEY (`orderDetailId`),
  UNIQUE KEY `unique_order_book` (`orderId`,`bookId`),
  KEY `idx_orderdetails_orderId` (`orderId`),
  KEY `idx_orderdetails_bookId` (`bookId`),
  CONSTRAINT `fk_orderdetails_orderId` FOREIGN KEY (`orderId`) REFERENCES
`orders` (`orderId`) ON DELETE CASCADE,
  CONSTRAINT `fk_orderdetails_bookId` FOREIGN KEY (`bookId`) REFERENCES
`books` (`bookId`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;


--
-- Table structure for table `shoppingcart`
--


CREATE TABLE `shoppingcart` (
  `username` varchar(50) NOT NULL,
  `bookId` int NOT NULL,
  `quantity` int NOT NULL DEFAULT 1,
  `price` double NOT NULL,
  `dateAdded` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`username`,`bookId`),
  KEY `idx_cart_bookId` (`bookId`),
  CONSTRAINT `fk_cart_username` FOREIGN KEY (`username`) REFERENCES `users`
(`username`) ON DELETE CASCADE,
  CONSTRAINT `fk_cart_bookId` FOREIGN KEY (`bookId`) REFERENCES `books`
(`bookId`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

```
/*!40103 SET TIME_ZONE=@OLD_TIME_ZONE */;

/*!40101 SET SQL_MODE=@OLD_SQL_MODE */;

/*!40014 SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS */;

/*!40014 SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS */;

/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;

/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;

/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;

/*!40111 SET SQL_NOTES=@OLD_SQL_NOTES */;


-- Database creation completed successfully!
```

Key points of Database Design:

- `PRIMARY KEY`: Each table has a primary key (`bookId`, `username`, `orderId`, `orderDetailId`, and a composite key for `shoppingcart`) to uniquely identify records.

- `FOREIGN KEY`: Relationships between tables are established using foreign keys (`username` in `orders` and `shoppingcart`, `orderId` and `bookId` in `orderdetails`, `bookId` in `shoppingcart`). These ensure referential integrity.

- `AUTO_INCREMENT`: `bookId`, `orderId`, and `orderDetailId` are set to auto-increment, simplifying record insertion.

- `DEFAULT` values: `orderDate` and `dateAdded` default to the current timestamp, status defaults to "`Pending`" in orders, and quantity defaults to `1` in `orderdetails` and `shoppingcart.totalAmount` and `totalItems` in `orders` also have default values.

- `GENERATED ALWAYS AS`: The subtotal column in `orderdetails` is a generated column, automatically calculating `quantity x price` and storing it. This ensures data consistency.

- `ON DELETE SET NULL` and `ON DELETE CASCADE`:

    - `ON DELETE SET NULL` on `fk_orders_username` means if a user is deleted, their `orderId` in the orders table will become `NULL` instead of deleting the order itself.

    - `ON DELETE CASCADE` is used for `orderdetails` and `shoppingcart` foreign keys. This means if an order or book is deleted, the corresponding detail records or cart entries will also be deleted.

- `Indexes`: Indexes are added to foreign key columns (`idx_orders_username`, `idx_orders_date`, `idx_orderdetails_orderId`, `idx_orderdetails_bookId`, `idx_cart_bookId`) to improve query performance.

- `unique_order_book`: A unique constraint is added to `orderdetails` on (`orderId`, `bookId`) to prevent duplicate entries for the same book within a single order.

3. SYSTEM IMPLEMENTATION

3.1. Technology Stack

Layer	Technology	Rationale
Database	MySQL 8.0	Robust relational database with excellent performance for ACID transactions, widespread enterprise adoption, and strong Spring Boot integration support
Backend	Java 22 and Spring Boot 3.2.0	Modern Java features with enhanced performance, Spring Boot's auto-configuration and dependency injection for rapid development, comprehensive ecosystem
Frontend	Thymeleaf and HTML5 /CSS3 /JavaScript	Server-side templating for dynamic content generation, seamless Spring Boot integration, responsive web design capabilities
Web Server	Apache Tomcat (Embedded)	Lightweight servlet container embedded in Spring Boot, simplified deployment and configuration
Build Tool	Maven 3.9.9	Standardized dependency management, automated build processes, and project lifecycle management
ORM Framework	Spring Data JPA	Object-relational mapping with Hibernate implementation, reducing boilerplate database code

Table 3. System Technology Stack Overview

3.2. Code Structure and Design Patterns

DAO Package

The Data Access Object (DAO) package implements a robust data access layer that abstracts database operations and provides a clean interface between the business logic and data persistence layers. It follows the DAO design pattern, which separates data access logic from business logic, ensuring maintainable and testable code architecture. The DAO package consists of five primary components, each responsible for managing data operations for specific entities:

- `BookDAO`: Handles all book-related database operations including inventory management.
- `UserDAO`: Manages user authentication, registration, and profile operations.
- `OrderDAO`: Processes order creation, retrieval, and status management.
- `OrderDetailDAO`: Manages individual order line items and product associations.
- `ShoppingCartDAO`: Handles shopping cart persistence and session management.

The DAO implementation demonstrates a sophisticated dual approach to database connectivity:

- Repository for *Book DAO*

```
@Repository
public class BookDAO {
    private final Database database;

    @Autowired
    private JdbcTemplate jdbcTemplate;
```

```
    public BookDAO(Database database) {
        this.database = database;
    }
}
```

This architecture supports both traditional JDBC operations through the custom `Database` class and Spring's `JdbcTemplate` for simplified database access, providing flexibility and backwards compatibility.

- Resource Management & Connection Handling

It demonstrates exemplary resource management practices with consistent try-finally blocks:

```
public Book getBookById(Integer bookId) throws SQLException {
    Connection con = null;
    PreparedStatement stm = null;
    ResultSet rs = null;
    try {
        String sql = "SELECT bookId, type, name, price, publisher,
quantity FROM books WHERE bookId = ?";
        con = database.getConnection();
        stm = con.prepareStatement(sql);
        stm.setInt(1, bookId);
        rs = stm.executeQuery();
        // Process results...
    } finally {
        if (rs != null) rs.close();
        if (stm != null) stm.close();
        if (con != null) con.close();
    }
}
```

This pattern ensures proper cleanup of database resources, preventing connection leaks and maintaining system stability under high load conditions.

- Dynamic Query Construction

The system implements sophisticated dynamic query building for flexible search operations, enables efficient database queries while maintaining security through parameterized statements:

```
public void fetchBooks(Integer bookId, String type, String name, String
publisher) throws SQLException {
    String sql = "SELECT bookId, type, name, price, publisher, quantity
FROM books WHERE 1=1";
    List<Object> params = new ArrayList<>();

    if (bookId != null && bookId > 0) {
        sql += " AND bookId = ?";
        params.add(bookId);
    }
    if (type != null && !type.trim().isEmpty()) {
        sql += " AND type LIKE ?";
        params.add("%" + type + "%");
    }
    // Additional conditions...
}
```

- CRUD Operations Implementation

Create Operations: The system implements robust creation methods with comprehensive validation.

```
public boolean createBook(Book book) throws SQLException {
    if (book == null) return false;

    if (book.getType() == null || book.getType().trim().isEmpty() ||
```

```
        book.getName() == null || book.getName().trim().isEmpty() ||
        book.getPrice() < 0 || book.getQuantity() < 0) {
        return false;
    }

    String sql = "INSERT INTO books (type, name, price, publisher,
quantity) VALUES (?, ?, ?, ?, ?)";
    // Implementation continues...
}
```

Update Operations with State Synchronization: The update operations include automatic cache synchronization to maintain data consistency.

```
public boolean updateBook(Book info) throws SQLException {
    // Validation and update logic...
    boolean result = stm.executeUpdate() > 0;

    // Cache synchronization
    if (result) {
        getAllBooks(); // Refresh local cache
    }

    return result;
}
```

- Security and Validation Features

Input Validation and Sanitization: The DAO layer implements multiple levels of input validation:

- *Null checking*: Prevents null pointer exceptions.
- *Business rule validation*: Ensures data integrity (e.g., non-negative prices).
- *String validation:* Handles empty and whitespace-only inputs.
- *Type safety*: Validates data types before database operations.

Authentication Security: The `UserDAO` implements secure authentication patterns with password handling considerations.

```
public User login(String username, String password) throws SQLException {
    String sql = "SELECT username, firstName, lastName, phoneNumber, role
FROM users WHERE username = ? AND password = ?";
    // Implementation includes role-based access control
    if (rs.next()) {
        temp.setPassword("*********"); // Security masking
        temp.setRole(rs.getString("role")); // Role assignment
    }
}
```

- Advanced Features and Optimizations

Inventory Management: `BookDAO` includes sophisticated inventory management capabilities, ensures atomic stock operations, preventing overselling scenarios common in e-commerce applications.

```
public boolean reduceBookStock(int bookId, int quantityToReduce) {
    String sql = "UPDATE books SET quantity = quantity - ? WHERE bookId =
? AND quantity >= ?";
    // Atomic stock reduction with availability checking
}
```

Error Handling and Logging

```
public Book getBookByIdSafe(Integer bookId) {
    try {
        return getBookById(bookId);
    } catch (SQLException e) {
        System.err.println("Error getting book by ID " + bookId + ": " +
e.getMessage());
        return null;
```

```
    }
}
```

MODEL Package

The Model package serves as the core data representation layer of the Bookstore Administration, implementing the domain objects that correspond directly to the database schema and business entities. This utilizes JPA annotations to establish object-relational mapping, enabling seamless translation between Java objects and database records while maintaining data integrity and consistency.

The package employs JPA annotations to create a robust object-relational mapping framework that simplifies database interactions while maintaining clean separation between data representation and business logic. Each entity class is annotated with `@Entity` and `@Table` annotations to establish clear mapping to database tables, while field-level annotations provide fine-grained control over column mapping and constraints.

`Book entity` demonstrates sophisticated JPA implementation with auto-generated primary keys and comprehensive field mapping:

```
@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "bookId";
    private Integer bookId;

    @Column(name = "type")
    private String type;
```

```
    @Column(name = "name")
    private String name;

    @Column(name = "price")
    private Double price;

    @Column(name = "publisher")
    private String publisher;

    @Column(name = "quantity")
    private Integer quantity;

    // Constructor with business logic validation
    public Book(String type, String name, Double price, String publisher,
Integer quantity) {
        this.type = type;
        this.name = name;
        this.price = price;
        this.publisher = publisher;
        this.quantity = quantity;
    }
}
```

`User entity` implements role-based access control through default value assignment and comprehensive user profile management:

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @Column(name = "username")
    private String username;

    @Column(name = "firstName")
    private String firstName;
```

```
    @Column(name = "lastName")
    private String lastName;

    @Column(name = "phoneNumber")
    private String phoneNumber;

    @Column(name = "password")
    private String password;

    @Column(name = "role")
    private String role;

    // Default constructor with business rule enforcement
    public User() {
        this.role = "customer"; // Default role assignment
    }

    public User(String username, String firstName, String lastName,
                String phoneNumber, String password, String role) {
        this.username = username;
        this.firstName = firstName;
        this.lastName = lastName;
        this.phoneNumber = phoneNumber;
        this.password = password;
        this.role = role != null ? role : "customer";
    }
}
```

`Order entity` showcases advanced temporal data management and automatic field initialization:

```
@Entity
@Table(name = "orders")
public class Order {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "orderId")
    private Integer orderId;

    @Column(name = "username", nullable = false)
    private String username;

    @Column(name = "orderDate")
    private LocalDateTime orderDate;

    @Column(name = "status")
    private String status;

    @Column(name = "totalAmount")
    private Double totalAmount;

    @Column(name = "totalItems")
    private Integer totalItems;

    // Automatic initialization with business defaults
    public Order() {
        this.orderDate = LocalDateTime.now();
        this.status = "Pending";
    }
}
```

The Model package implements sophisticated entity relationships through composite key structures and transient field management. The `ShoppingCart entity` utilizes `@IdClass` annotation for composite primary key handling, while the `OrderDetails entity` incorporates transient fields for enhanced data presentation:

```
@Entity
@Table(name = "shoppingcart")
@IdClass(ShoppingCartId.class)
public class ShoppingCart {
```

```
    @Id
    @Column(name = "username")
    private String username;

    @Id
    @Column(name = "bookId")
    private Integer bookId;

    @Column(name = "quantity")
    private Integer quantity;

    @Column(name = "price")
    private Double price;
}
```

`OrderDetails` `entity` demonstrates advanced JPA features including precision-controlled decimal fields and transient properties for runtime data enhancement:

```
@Entity
@Table(name = "orderdetails")
public class OrderDetails {
    @Column(name = "subtotal", nullable = false, precision = 10, scale =
2)
    private BigDecimal subtotal;

    // Transient fields for enhanced presentation
    @Transient
    private String bookName;

    @Transient
    private String bookPublisher;

    // Business logic integration
    public OrderDetails(Integer orderId, Integer bookId, Integer
quantity, Double price) {
        this.orderId = orderId;
```

```
        this.bookId = bookId;
        this.quantity = quantity;
        this.price = price;
        this.subtotal = BigDecimal.valueOf(quantity * price);
    }
}
```

The Model package emphasizes type safety through consistent use of wrapper classes (`Integer`, `Double`) rather than primitive types, enabling proper null handling and database compatibility. Each entity implements comprehensive `toString()` methods for debugging and logging purposes, while maintaining proper encapsulation through private fields and public accessor methods. The package design supports both automatic field generation through JPA annotations and manual business logic implementation through custom constructors, providing flexibility for different use cases while maintaining data consistency across the application.

CONTROLLER Package

The Controller package represents the presentation layer's core component, implementing the MVC pattern to handle HTTP requests, coordinate business logic execution, and manage response generation. This package serves as the primary interface between external clients and the application's business logic, ensuring proper request routing, input validation, and response formatting.

This package is structured around Spring Boot's annotation-driven architecture, utilizing RESTful design principles and dependency injection patterns. Each controller class is specialized for specific functional domains, promoting single responsibility principle and enabling modular development. The package implements both traditional MVC controllers for server-side rendered pages and REST API controllers for AJAX and mobile client interactions.

The controllers leverage Spring's `@Controller` and `@RestController` annotations to define their behavior, with `@RequestMapping` annotations providing fine-grained control

over URL routing and HTTP method handling. Dependency injection through `@Autowired` annotations ensures loose coupling with DAO components while maintaining clean separation between presentation and data access concerns.

`BookApiController` exemplifies the package's REST API implementation, providing comprehensive book management endpoints. This controller demonstrates advanced Spring Boot techniques including path variable extraction, query parameter handling, and comprehensive error management:

```
@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*")
public class BookApiController {

    @Autowired
    private BookDAO bookDAO;

    @GetMapping("/books/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Integer id) {
        logger.info("Received request for book with id: {}", id);
        try {
            bookDAO.getAllBooks();
            List<Book> books = bookDAO.getBooksList();

            if (books != null) {
                Book book = books.stream()
                    .filter(b -> id.equals(b.getBookId()))
                    .findFirst()
                    .orElse(null);

                if (book != null) {
                    return ResponseEntity.ok(book);
                }
            }
```

```
            return ResponseEntity.notFound().build();
        } catch (Exception e) {
            logger.error("Error fetching book with id: {}", id, e);
            return
ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
        }
    }
}
```

`UserCartController` demonstrates sophisticated state management by supporting both guest (session-based) and authenticated user (database-persisted) shopping carts with seamless synchronization capabilities:

```
@PostMapping("/sync")
public ResponseEntity<Map<String, Object>> syncGuestCart(
    @RequestBody List<Map<String, Object>> guestCart, HttpSession
session) {

    String username = (String) session.getAttribute("username");
    if (username == null) {
        return ResponseEntity.badRequest().body(
            Map.of("success", false, "message", "Please login first"));
    }

    try {
        int syncedItems = 0;
        for (Map<String, Object> guestItem : guestCart) {
            // Complex synchronization logic with existing cart items
            List<ShoppingCart> existingItems =
shoppingCartDAO.getItems();
            ShoppingCart existingItem = existingItems.stream()
                .filter(cartItem -> cartItem.getBookId().equals(bookId))
                .findFirst()
                .orElse(null);

            if (existingItem != null) {
```

```
                // Merge quantities for existing items
                int newQuantity = existingItem.getQuantity() + quantity;
                existingItem.setQuantity(newQuantity);
                if (shoppingCartDAO.updateItem(existingItem)) {
                    syncedItems++;
                }
            } else {
                // Add new items to user cart
                if (shoppingCartDAO.addItem(username, bookId, quantity,
price)) {
                    syncedItems++;
                }
            }
        }

        return ResponseEntity.ok(Map.of(
            "success", true,
            "message", "Guest cart synced successfully",
            "syncedItems", syncedItems
        ));
    } catch (Exception e) {
        logger.error("Error syncing guest cart: {}", e.getMessage());
        return ResponseEntity.badRequest().body(
            Map.of("success", false, "message", "Error syncing guest
cart"));
    }
}
```

`AdminController` implements role-based access control with comprehensive session validation, demonstrating enterprise-level security patterns:

```
@PostMapping("/admin/updateStock")
@ResponseBody
public ResponseEntity<Map<String, Object>> updateStock(
    @RequestBody Map<String, Object> requestData, HttpSession session) {
```

```
    try {
        // Role-based authorization check
        String role = (String) session.getAttribute("role");
        if (!"admin".equals(role)) {
            return ResponseEntity.status(403).body(Map.of(
                "success", false,
                "message", "Access denied"
            ));
        }

        // Input validation and type safety
        Object bookIdObj = requestData.get("bookId");
        Object quantityObj = requestData.get("quantity");

        if (bookIdObj == null || quantityObj == null) {
            return ResponseEntity.badRequest().body(Map.of(
                "success", false,
                "message", "Missing required fields"
            ));
        }

        int bookId = Integer.parseInt(bookIdObj.toString());
        int quantity = Integer.parseInt(quantityObj.toString());

        // Business logic execution with data refresh
        boolean updated = bookDAO.updateBookStock(bookId, quantity);
        if (updated) {
            bookDAO.getAllBooks(); // Refresh cache
            return ResponseEntity.ok(Map.of(
                "success", true,
                "message", "Stock updated successfully"
            ));
        }

    } catch (NumberFormatException e) {
        return ResponseEntity.badRequest().body(Map.of(
            "success", false,
            "message", "Invalid number format"
```

```
        ));
    }
}
```

The Controller package implements comprehensive error handling through try-catch blocks that provide detailed logging for debugging while returning user-friendly error messages, utilizing SLF4J with appropriate log levels (INFO, WARN, ERROR) to support both development debugging and production monitoring requirements. Additionally, controllers maintain consistent response formats using `ResponseEntity<>` with standardized JSON structures containing success indicators, descriptive messages, and relevant data payloads, which facilitates frontend integration and provides predictable API behavior for client applications.

User Repository Interface

`UserRepository` defines a Spring Data JPA repository interface that provides automated database operations for User entities without requiring manual SQL code:

```
@Repository
public interface UserRepository extends JpaRepository<User, String> {
    Optional<User> findByUsername(String username);
    Optional<User> findByUsernameAndPassword(String username, String
password);
}
```

Application Bootstrap

`ServletInitializer` provides deployment flexibility by extending `SpringBootServletInitializer`, enabling the application to be packaged as a WAR file and deployed on traditional application servers like Tomcat or Jetty, rather than relying

solely on Spring Boot's embedded server. Bootstrap constructs infrastructure components that handle technical application startup and deployment concerns without containing any bookstore-specific business logic such as inventory management, user authentication, or shopping cart functionality.

```
public class ServletInitializer extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder
application) {
        return
application.sources(Bookstoremanagementsystemversion2Application.class);
    }

}
```

4. USER INTERFACE (UI) & USER EXPERIENCE (UX) DESIGN

4.1. UI Design Principles

This section provides an analysis of the web application's user interface, focusing on its layout structure, navigational clarity, responsiveness, and user experience across various functional pages. The design reflects a balance between aesthetic appeal and functional usability.

- *Responsive Layout:* The use of max-width, padding, and grid layouts (e.g., in *checkout.html*, *cart.css*, *admin.css*) suggests a design that adapts well to different screen sizes.

- *Clear Navigation:* A prominent, consistent navigation bar (.navbar in styles.css) is present across all main pages (*index.html*, *cart.html*, *my_orders.html*, *admin.html*),

featuring a logo, navigation links, and dynamic elements for authentication (*login*/*register*/*logout*) and admin access.

- *Intuitive Product Display:* Books are displayed in a clean, card-based layout (*.book_card* implied by *books.js* and *styles.css* context), allowing for easy browse, searching, and filtering.

- *Streamlined Shopping Experience:* The cart (*cart.html*, *cart.css*) is well-organized with item details, quantity controls, and a clear summary. The checkout process (*checkout.html*, *checkout.js*) is laid out logically with form fields for shipping and payment, along with an order summary.

- *User Account Management:* Dedicated pages for login (*login.html*) and registration (*register.html*) use a consistent, visually distinct form style with background gradients and prominent forms. "My Orders" (*my_orders.html*) provides a simple, card-based overview of past purchases, with "Order Details" (*order_details.html*) offering a detailed breakdown.

- *Feedback and Confirmation:* The UI provides immediate user feedback through success/error messages on forms and "toast" notifications (from *books.js*, *cart.js*) for actions like adding to cart. A prominent "Order Successful" page (*orderSuccess.html*, *orderConfirmation.html*) with a celebratory visual confirms completed transactions.

- *Administrative Interface:* A separate admin dashboard (*admin.html*, *admin.css*) provides structured tables and forms for managing users, books, and orders, using distinct styling to differentiate it from the customer-facing site.

4.2. Mockups Interface

The following section presents a demonstration of the project, showcasing key features, user interactions, and the overall functionality of the system. This demo illustrates how the application operates in practice, from both user and administrator perspectives.

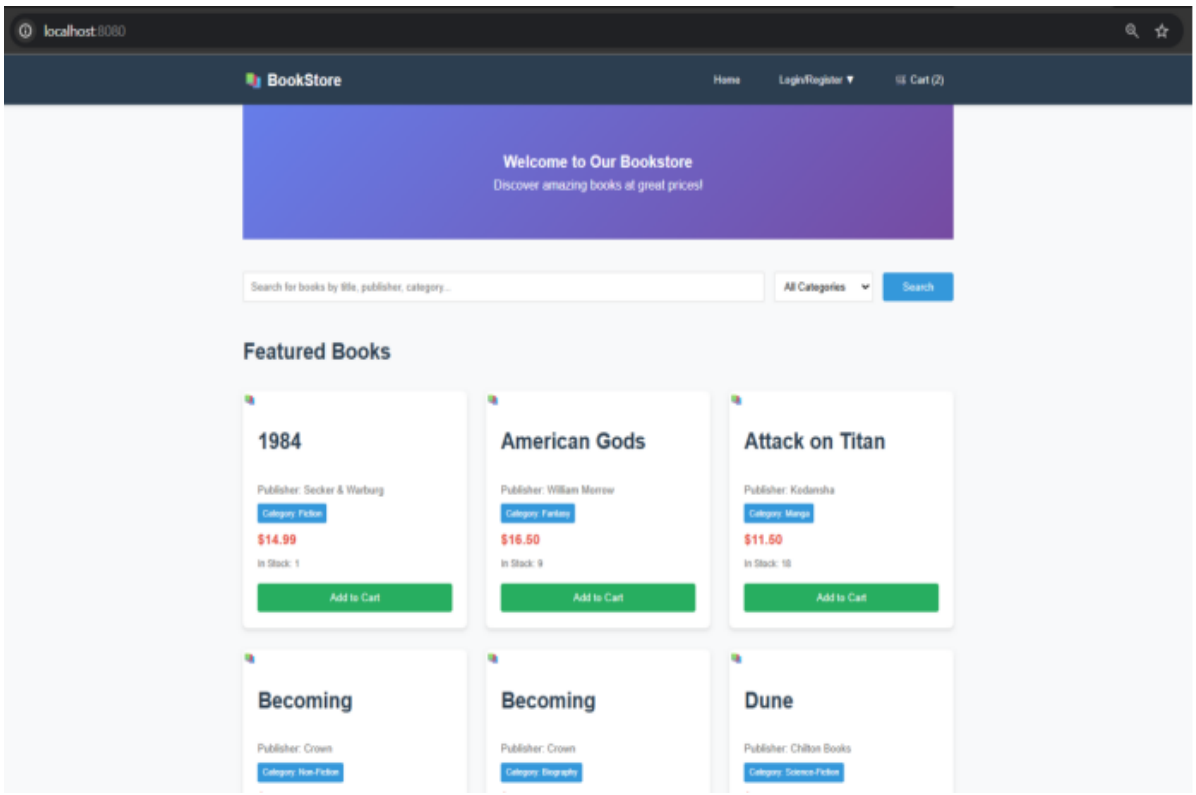

Image 4. Homepage System

HOME PAGE & BROWSE

- Initial Impression: The home page presents a clean, modern layout with a prominent navigation bar at the top, featuring a "BookStore" logo, "Home" link, and authentication options ("Login/Register ▼" dropdown).

- Content Discovery: Books are displayed in a grid format with clear titles, authors, prices, and "*Add to Cart*" buttons. A search bar and a "*Type Filter*" dropdown are presented to easily find specific books or browse by category.

- *Call to Action:* The "*Add to Cart*" button on each book card is the primary interaction point for purchasing.

- *User Feedback:* Clicking "*Add to Cart*" triggers a subtle "toast" notification confirming the addition, and the cart count in the navigation would update, providing immediate, non-intrusive feedback.

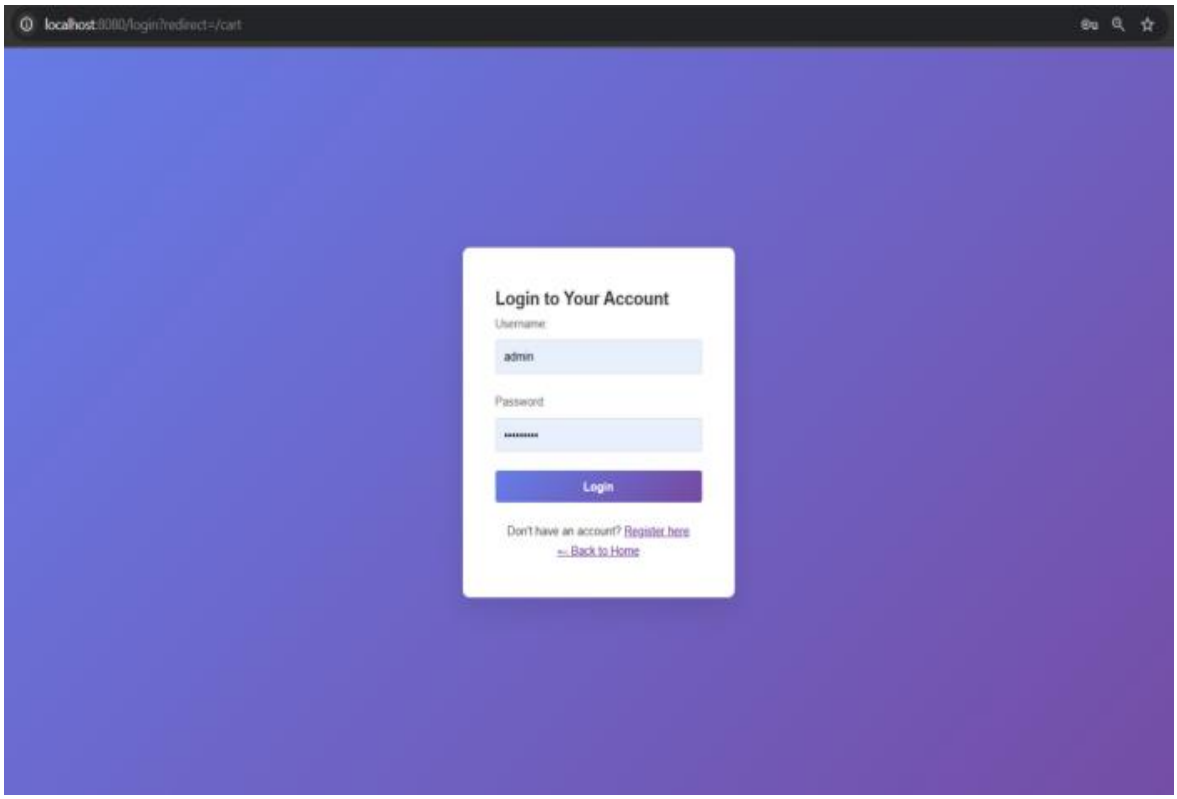

Image 5. User Identification

AUTHENTICATION CREDENTIALS

- *Simplified Focus:* The login page features a prominent, centralized form against a gradient background, minimizing distractions. It clearly asks for "*Username/Email*" and "*Password*".

- *Guided Action:* A "*Login*" button is the primary call to action. A "*Don't have an account? Register here*" link guides new users.

- *Feedback:* Input fields likely show validation (e.g., red borders for errors), and messages below the form would indicate success or failure of the login attempt.

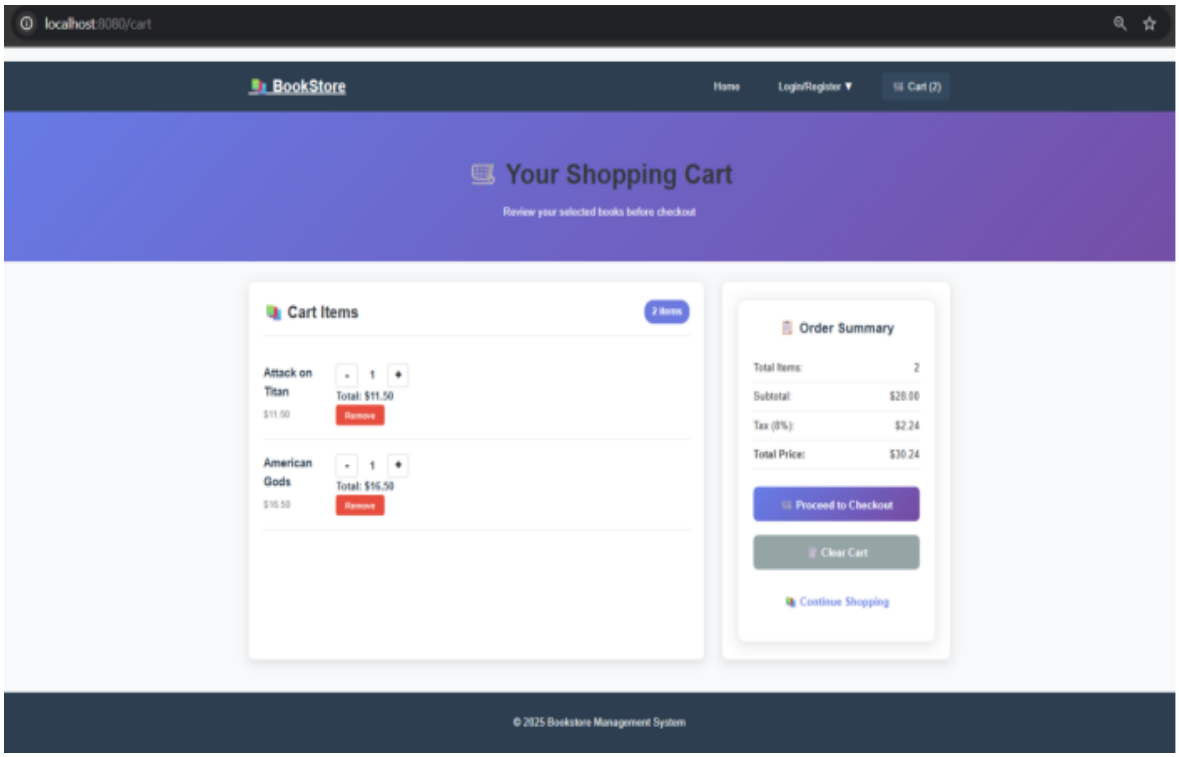

Image 6. User's Cart Details

SHOPPING CART

- *Overview & Control:* The cart page clearly lists items with their image, name, quantity, and price. Quantity can be easily adjusted using "+" and "-" buttons, and items can be removed.

- *Transparency:* A dedicated "Order Summary" section clearly shows the subtotal, tax, and total amount, providing financial clarity before checkout.

- *Key Action:* A prominent "Proceed to Checkout" button guides the user to finalize the purchase. A "Clear Cart" button allows for easy reset.

- *Conditional Guidance:* If a user isn't logged in when they click "Proceed to Checkout," a modal would likely appear, prompting them to log in or register, ensuring a seamless transition for order tracking.

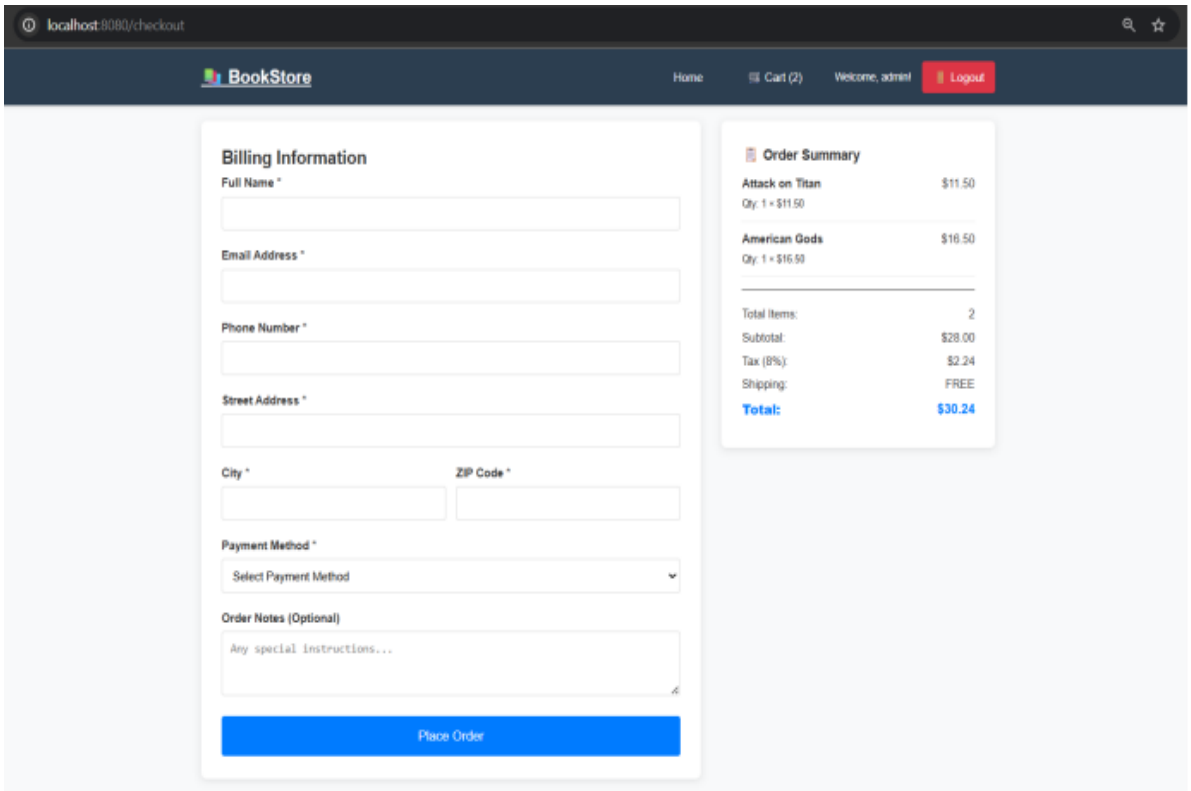

Image 7. Input Shipping Details

CHECKOUT PROCESS

- *Structured Progression:* The checkout page is divided into "*Shipping Information*" and "*Payment Information*" forms, alongside a final "Order Summary" on the right.

- *Information Input:* Clear labels and input fields for name, address, email, and payment details simplify data entry.

- *Confirmation:* The "*Order Summary*" reiterates the items and total, reinforcing what the user is about to purchase.

- *Final Action:* The "*Place Order*" button is the definitive step to complete the transaction.

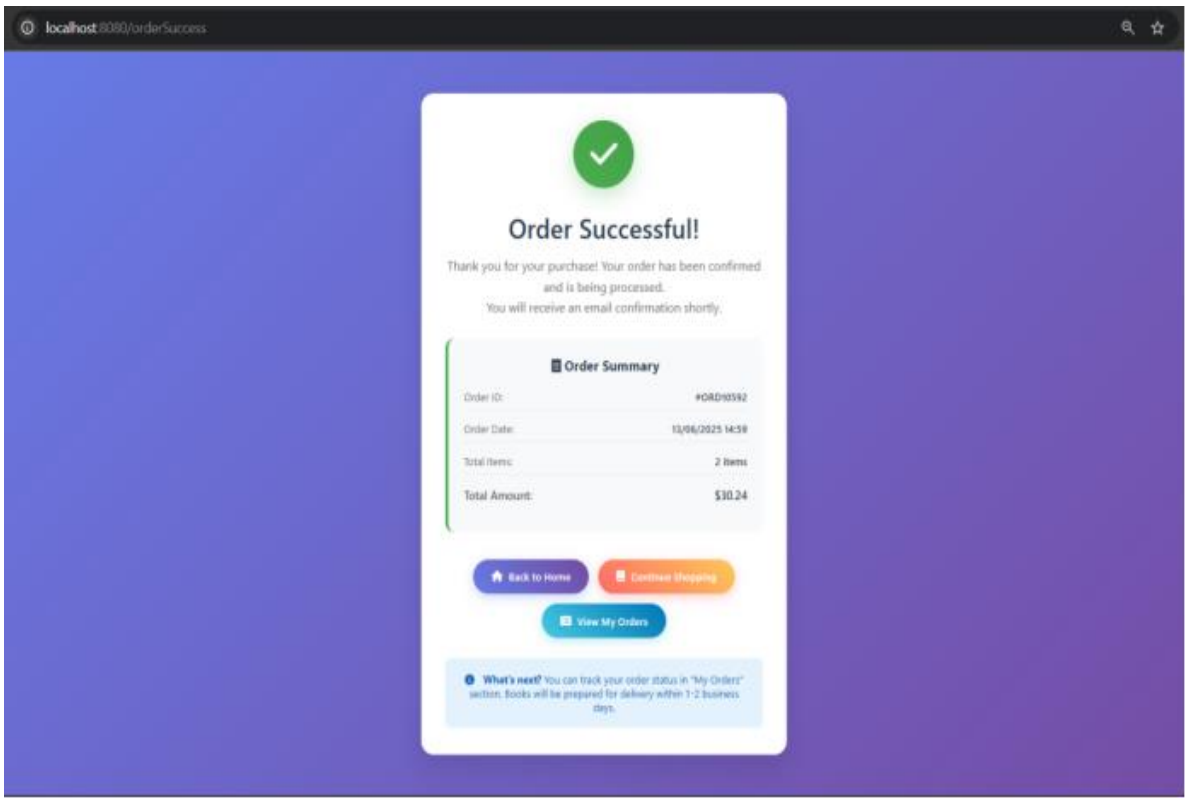

Image 8. Order Success Message

ORDER CONFIRMATION

- *Positive Reinforcement:* This page provides positive feedback with a large green checkmark, "Order Successful!" title, and a congratulatory message.

- *Essential Details:* Key information like Order ID, number of items, and total amount are immediately visible, confirming the transaction.

- *Clear Next Steps:* Action buttons such as "*Back to Home*", "*Continue Shopping*" and "*View My Orders*" immediately guide the user on what to do next, reducing potential confusion. An additional info section provides helpful guidance on tracking the order.

- *Delightful Experience:* The green color and visual elements contribute to a celebratory feel, and the previous code analysis mentioned a "confetti" effect, which would add a layer of delight to the user's successful purchase.

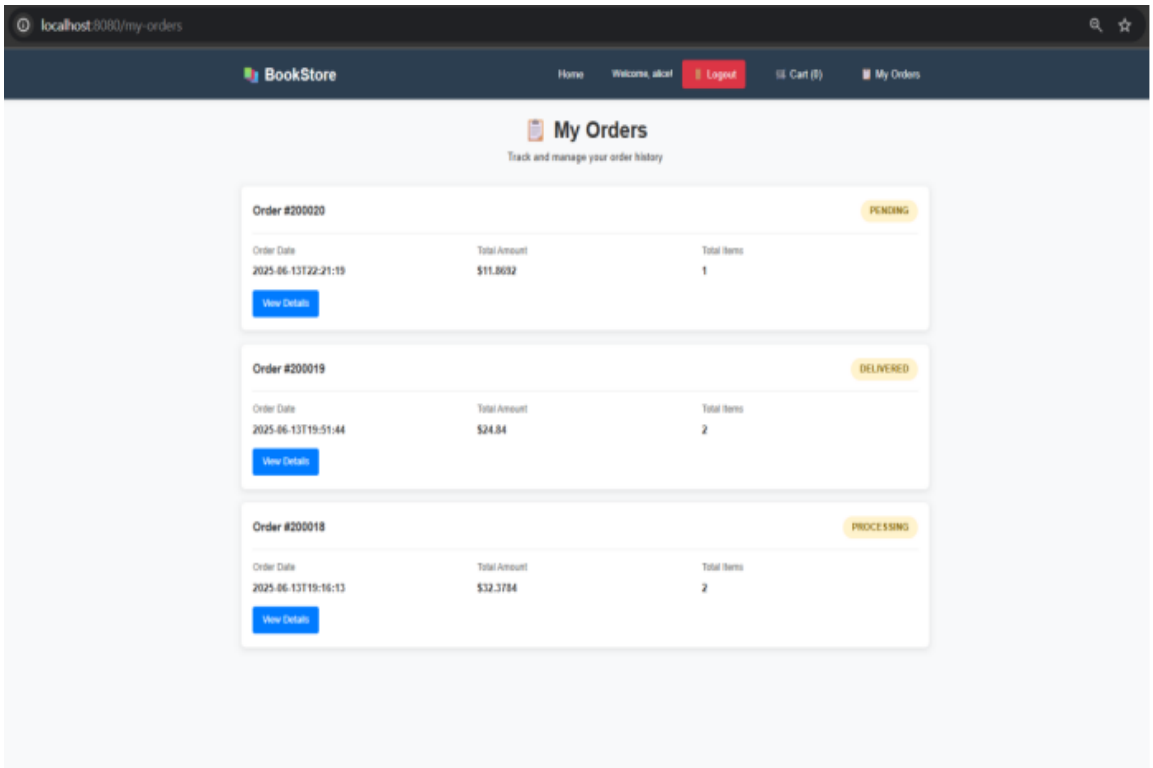

Image 9. Order Status View

MY ORDERS

- *Overview of Past Activity:* This page provides a clear list of a user's past orders, each presented as a "*card*" with basic details like Order ID, Date, and Status.

- *Easy Access to Details:* The card-like presentation with a hover effect suggests that each order is clickable, leading to more detailed information.

- *Status Clarity:* The "*Status*" is visually highlighted with badges (e.g., "*Shipped*"), allowing users to quickly understand the state of their orders.

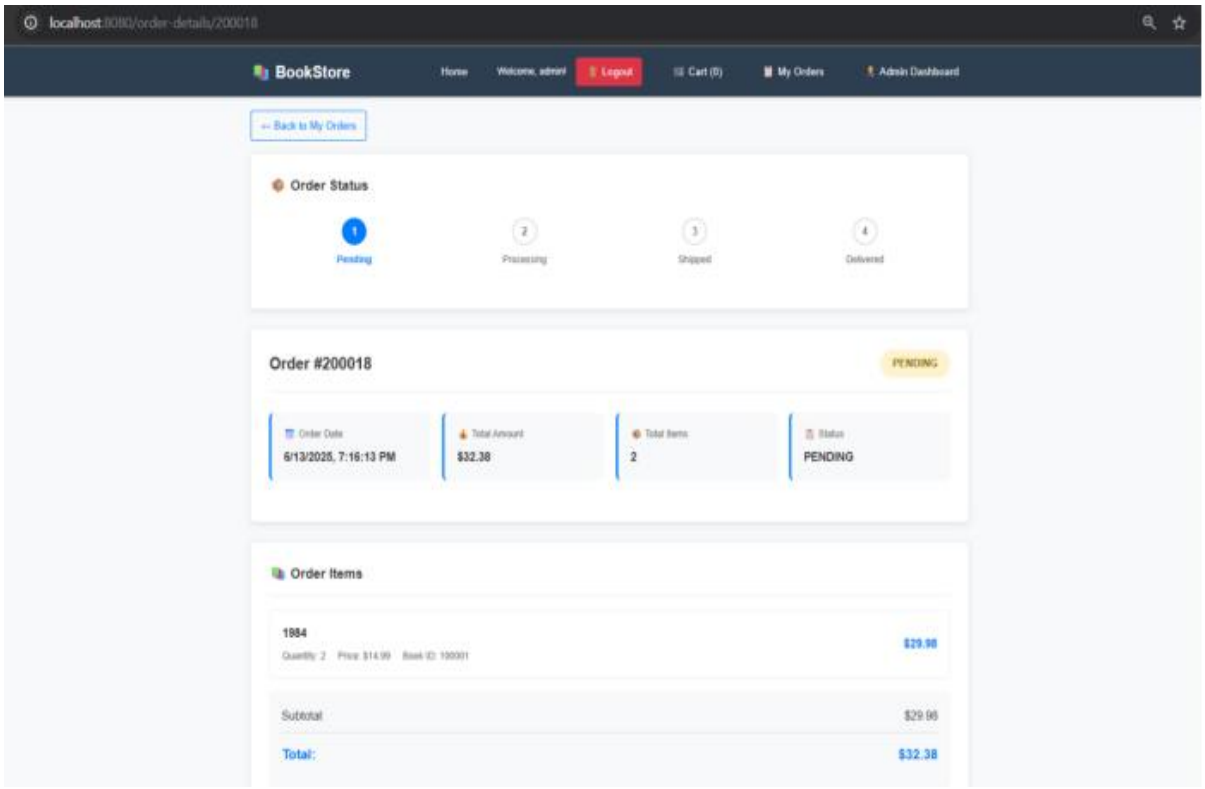

Image 10. Order Details View

ORDER DETAILS

- *Detailed Transparency:* This page provides a comprehensive breakdown of a specific order, including shipping information, payment method, the full list of items purchased (with quantity and price), and the total.

- *Navigational Cue:* A "*Back to Orders*" link is prominently placed at the top, allowing users to easily return to their full order history.

5. SETUP GUIDELINES

a) Tested Hardware Configurations:

- *Processors*: AMD RYZEN 7 6000 series, Intel Core i5/i7.

- *RAM*: Minimum 8GB (16GB recommended for optimal performance).
- *Storage*: 500GB SSD or equivalent (minimum 5GB free space for project).
- *Network*: Stable internet connection for dependency downloads.

b) Tested Operating System: Window 11, macOS Ventura, Ubuntu 20.04+.

c) Required Software Version:

Core Requirements:

- *Java Development Kit (JDK)*: Version 21+ (JDK 22 tested and recommended).
- *MySQL Server*: Version 8.0+.
- *Apache Maven*: Version 3.9.9.

Development Tools:

- *IDE*: Visual Studio Code, IntelliJ IDEA, or Eclipse.
- *Version Control*: Git (for GitHub integration).
- *Database Management*: MySQL Workbench.
- *Express Setup Commands*: Terminal Command Prompt.

d) Application Configuration:

Update file `src/main/resources/application.properties`.

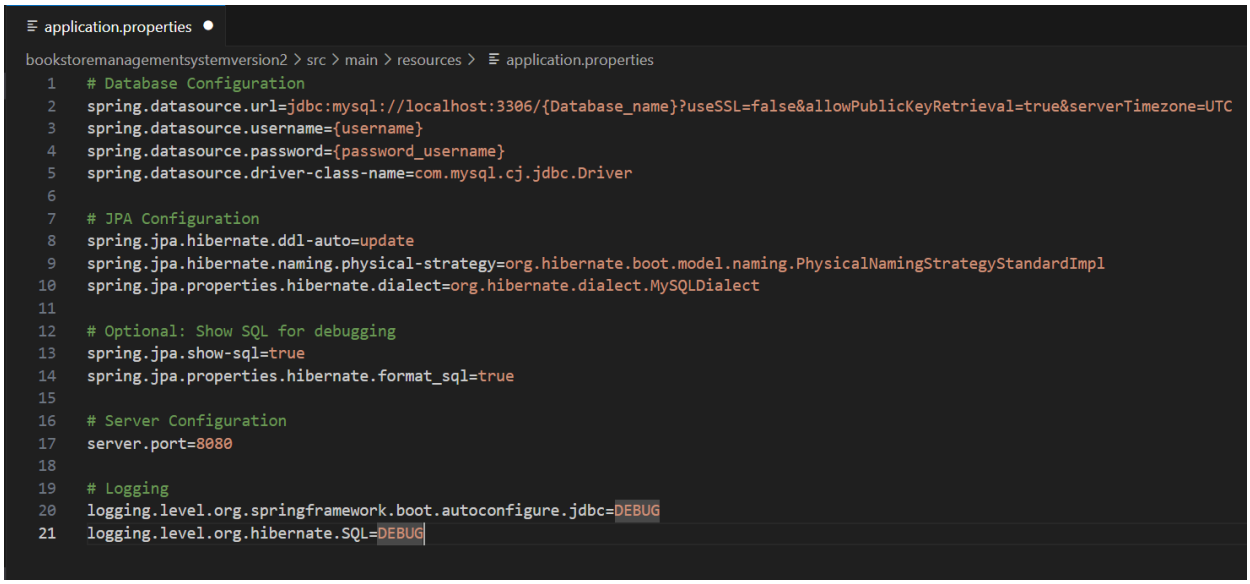

Image 11. MySQL Application Configuration

Replace the brackets {*Database_name*}, {*username*} and {*password_username*} according to your MySQL user account.

Verify Tool Installations:

```
mvn --version
java --version
mysql -version
```

Database Setup:

```
mysql -u {user_name} -p{user_password} -e "DROP DATABASE IF EXISTS
bookstore; CREATE DATABASE bookstore;"
mysql -u {user_name} -p{user_password} bookstore < bookstore.sql
# Verify the imported file
mysql -u {user_name} -p{user_password} bookstore -e "SELECT COUNT(*) as
users FROM users; SELECT COUNT(*) as books FROM books;"
```

Build and Run Application:

```
cd                  {System_directory}\bookstoremanagementsystemversion2
taskkill                /F                  /IM                 java.exe
mvnw.cmd                          clean                              install
mvn clean spring-boot:run
```

Access Application: Open your browser and navigate to http://localhost:8080.To log in to the system as the "Admin" role, using the username "admin" and password "adminpass".

6. CONCLUSION & FUTURE WORK

6.1. Summary of Achievement & Knowledge

The development of the Bookstore Administration system provided a comprehensive learning experience across the full software development lifecycle. It involved designing a full-stack web application with attention to software architecture, code modularity, and system scalability. The project emphasized practical application of theoretical concepts, including system decomposition, layered design, and RESTful communication.

Spring Boot was utilized to implement a robust backend infrastructure, incorporating the Model-View-Controller (MVC) pattern, dependency injection, and Data Access Objects (DAOs). MySQL was used for relational database design, supporting core business logic such as authentication, inventory management, and order processing. Structured Query Language (SQL) scripts were employed to initialize and maintain the persistence layer effectively.

On the client side, HTML, CSS, and JavaScript were used to build responsive and user-friendly interfaces for both customers and administrators. Pages such as login, registration, cart, checkout, and dashboards reflected a solid understanding of frontend development and user experience (UX) principles. Integration with backend APIs facilitated dynamic content rendering and asynchronous interactions.

In addition, the project introduced essential DevOps practices. Git was used for source control and collaborative development, while Maven handled dependency management and build automation. Agile methodologies guided team coordination, sprint planning, and iterative implementation. Overall, the project enhanced technical proficiency and demonstrated readiness for real-world software engineering challenges.

6.2. Challenges Encounters & Solutions

While the development of the Bookstore Administration demonstrates significant technical accomplishments, it is crucial to acknowledge potential limitations and challenges that may arise in both current implementation and future scalability. A comprehensive risk analysis allows the development team to proactively identify areas of improvement, prioritize resource allocation, and ensure long-term system robustness. The following table summarizes the key challenges identified in the current system, assesses their potential impact, and proposes targeted mitigation strategies to address each risk effectively.

Challenges	Description	Proposed Solution
Security Vulnerabilities	Lack of advanced security measures (password hashing, role-based access, protection against injections and cross-site attacks).	Implement Spring Security; apply bcrypt/Argon2 hashing; sanitize user inputs; enforce HTTPS; set secure HTTP headers; apply RBAC for granular access control.
Scalability Limitations	Current monolithic design may struggle under increasing user load and complex business growth.	Adopt microservices architecture; containerize with Docker; orchestrate using Kubernetes; introduce caching layers (Redis); optimize DB queries.
Missing Business Intelligence & Analytics	No support for user behavior analytics or business performance tracking	Integrate analytics tools (Google Analytics, Tableau); develop custom admin dashboards for KPIs, sales trends, and customer insights.
Lack of Automated Testing	Absence of systematic unit, integration, and UI tests	Establish a comprehensive testing framework using JUnit, Mockito,

	increases maintenance risks.	Selenium/Cypress; implement CI/CD pipelines via Jenkins or GitHub Actions.
Insufficient Error Handling and Logging	Rudimentary error responses hinder debugging and degrade user experience.	Implement centralized exception handling (@ControllerAdvice); integrate logging frameworks (Logback, Log4j); design user-friendly error pages.
Internationalization & Accessibility Gaps	Limited language, currency, and accessibility support reduces global usability.	Incorporate i18n frameworks; currency conversion APIs; adhere to WCAG standards; support multiple locales dynamically.

Table 4. Risk Assessment

In summary, the development of the Bookstore Administration has provided a comprehensive platform for applying theoretical knowledge to a practical, full-stack software development project. Throughout the design and implementation phases, the team successfully applied modern software engineering methodologies, integrating backend, frontend, database, and system architecture components into a cohesive and functional application. In addition to the technical achievements, the project fostered valuable collaborative, problem-solving, and project management skills, which are critical for future professional practice.

The accompanying risk assessment has further highlighted key limitations and challenges that the system may encounter as it scales. By acknowledging these risks and proposing targeted solutions, a clear roadmap for future enhancements has been established, ensuring the system's adaptability, security, and robustness in evolving business and technological environments. This project not only demonstrates the team’s current technical capabilities but also establishes a strong foundation for continuous learning and improvement in complex software development endeavors.

Appendices